Contents

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

How Mobile Match Algeria picks the best 3 plans for a user — in plain words, with the research each part comes from, a fully worked example, and ready answers for the jury.

---

1. The idea, in three sentences

The user gives their **budget**, their **data need**, and **ranks** what matters most (price, data). The engine throws away every plan that doesn't fit the hard limits, then ranks the rest by **how close each plan is to the "dream" plan**. It uses no AI and no other users' data, only this user's profile, so it works from day one.

---

2. How it works — five steps

1. **Filter:** drop any plan over budget, or with the wrong operator / type / network.
2. **Weights:** a fixed formula (Rank-Order Centroid) turns the user's ranking into numbers — price #1, data #2 → **0.75 / 0.25**. (We don't pick these; see §7.)
3. **Two reference plans:** the **dream** (cheapest *and* most data) and the **nightmare** (priciest, least data).
4. **Closeness:** measure how near each plan is to the dream versus the nightmare — a score from **0 (worst) to 1 (best)**.
5. **Rank by closeness**, show the **top 3** (the score becomes the match %).

---

3. The one formula to remember

$$\text{closeness } C = \frac{\text{(distance to the nightmare plan)}}{\text{(distance to the dream)} + \text{(distance to the nightmare)}}$$

The "distance" is a plain **straight-line distance** between two plans on a price-vs-data graph. A plan sitting on the dream gets C = 1; one sitting on the nightmare gets C = 0.

---

4. Where each part comes from (the research)

Nothing here is invented — each step is a published method:

Part of the engine	What it does	Comes from (research)
Hard filters	remove plans that don't fit	**Conjunctive (constraint) method** — non-compensatory screening (**Hwang & Yoon, 1981**)
Weights from a ranking	"price #1, data #2" → 0.75 / 0.25	**Rank-Order Centroid — Barron & Barrett (1996)**
Ranking by closeness	the formula in §3	**TOPSIS — Hwang & Yoon (1981)**; recent guide: **Taherdoost & Madanchian (2023)**

What is *ours* are setup choices, not formulas: which attributes are filters vs ranked criteria (price, data), that the budget is a hard ceiling, and two small conventions ("unlimited" counts as the best value; equal weights if the user ranks nothing). None of these computes a score — the scoring is entirely the cited methods.

---

5. A worked example — with the full numbers

Setup: **budget 2000 DA**, **need 10 GB**, user ranks **price #1, data #2**. Three plans pass the budget filter:

Plan	Price	Data
A	1000 DA	8 GB
B	1100 DA	15 GB
C	600 DA	4 GB

Step 1 — turn each plan into two comparable scores, then weight them. Dinars and GB can't be compared directly, so each column is rescaled into fractions (square the values, add them, take the square root, then divide — the computer does this), and each fraction is multiplied by its weight (price × 0.75, data × 0.25). Each plan becomes **two numbers** — picture it as a dot on a graph:

Plan	price-score	data-score
A	0.468	0.114
B	0.515	0.215
C	0.281	0.057

Step 2 — the dream and nightmare dots (price: lower is better; data: higher is better):

- **Dream** = cheapest price-score (0.281, from C) + most data-score (0.215, from B) → **(0.281, 0.215)**
- **Nightmare** = priciest (0.515, from B) + least data (0.057, from C) → **(0.515, 0.057)**

Step 3 — for each plan, the straight-line distance to each dot (gap on price squared + gap on data squared, then square root), then `C = nightmare ÷ (dream + nightmare)`:

```
C  (0.281, 0.057):
   to dream     = sqrt( 0^2             + (0.057−0.215)^2 ) = sqrt(0.025) = 0.158
   to nightmare = sqrt( (0.281−0.515)^2 + 0^2             ) = sqrt(0.055) = 0.234
   closeness    = 0.234 / (0.158 + 0.234) = 0.60     -> 1st
```

```
B  (0.515, 0.215):
   to dream     = sqrt( (0.515-0.281)^2 + 0^2             ) = 0.234
   to nightmare = sqrt( 0^2             + (0.215-0.057)^2 ) = 0.158
   closeness    = 0.158 / (0.234 + 0.158) = 0.40     -> 2nd

A  (0.468, 0.114):
   to dream     = sqrt( (0.468-0.281)^2 + (0.114-0.215)^2 ) = sqrt(0.045) = 0.212
   to nightmare = sqrt( (0.468-0.515)^2 + (0.114-0.057)^2 ) = sqrt(0.005) = 0.074
   closeness    = 0.074 / (0.212 + 0.074) = 0.26     -> 3rd
```

Result: C → B → A. C is the cheapest, so on price it sits *exactly on the dream* (distance 0) and price weighs most → highest closeness, 0.60. B is best on data but is the priciest → 0.40. A is middling and sits close to the nightmare → 0.26. If the user ranked **data first**, the 0.75 weight would shift to data and **B** would jump to the top.

---

6. The files

- **recommendation.ts** — the engine: filters, `rocWeights()`, `topsis()`.
- **api/recommendations/route.ts** — the API (validated, rate-limited).
- **recommend/page.tsx** — the form + ranked cards (with savings).

---

7. Jury questions — ready answers

- **What are you calculating? A straight-line (Euclidean) distance?** → Yes. Each plan's distance to the dream and the nightmare plan, combined into a closeness score (TOPSIS).
- **Why are the weights 0.75 and 0.25, not 0.70 / 0.30?** → We don't pick them. The **Rank-Order Centroid** formula turns a *ranking* into weights, and for two ranked criteria it always gives exactly 0.75 and 0.25. Choosing 0.70/0.30 would be inventing numbers — ROC avoids that.
- **Did you invent the formula?** → No. Filtering = conjunctive method, weights = ROC, ranking = TOPSIS — all published (Hwang & Yoon 1981; Barron & Barrett 1996). Only the setup choices (which attributes are filters vs criteria, the budget ceiling) are ours.
- **Does it work with no users?** → Yes, it's content-based (plan attributes + the user's profile), so there's no cold-start problem.
- **Why TOPSIS, not AI / a weighted sum / lexicographic / AHP?** → AI needs a usage history we don't have; a weighted sum is unbounded and scale-sensitive; strict lexicographic is too rigid; AHP needs many pairwise comparisons. TOPSIS is bounded, content-based, and pairs naturally with the ranking-based weights.
- **What if no plan fits?** → The list is empty (the budget ceiling is strict); the user can raise the budget or relax a filter.

Next guide: the search engine (token parser + FTS5). Tell me when you want it.